BACKEND DEVELOPER INTERVIEW WORKBOOK

First-round preparation • Python • REST APIs • Microservices • Java/Spring Boot • SQL

Goal: revise the highest-yield concepts and practise explaining your code clearly in a hands-on interview.

Interview clue: the example specifically asks you to reverse words without using `split()`. Expect the interviewer to care about loops, indexes, edge cases, and your reasoning—not just the final output.

How to use this workbook tonight

• **First 45 minutes:** solve the string coding exercise and the short coding drills without looking at answers.

• **Next 60 minutes:** revise Python frameworks, REST, microservices, and databases.

• **Next 45 minutes:** revise Java/Spring Boot and prepare concise project explanations.

• **Final 20 minutes:** answer the mock interview questions aloud; keep answers structured and specific.

Do not memorise every sentence. Learn the concepts and explain them in your own words.

1. Last-minute priority map

Priority	Topics	What you must be able to do
Must know	Strings, loops, lists, dicts, sets	Write and explain code without relying on shortcuts
Must know	REST API fundamentals	Explain methods, status codes, validation, errors
Must know	Flask / Django / FastAPI	Compare their purpose and request lifecycle
Must know	SQL	Write JOIN, GROUP BY, HAVING, and basic subqueries
High	Microservices	Explain service boundaries, failures, retries, observability
High	Spring Boot	Explain DI, controller/service/repository, JPA, transactions
High	Your projects	Explain architecture, your contribution, trade-offs, impact

A reliable approach to any coding question

• Restate the problem and confirm the exact expected output.

• Ask about empty input, punctuation, capitalization, repeated spaces, and constraints.

• Describe a simple approach before coding.

• Use meaningful variable names and avoid clever one-liners.

• Walk through one example manually.

• State time and space complexity, then test edge cases.

Your 60-second introduction — adapt this

> “I’m a backend developer at Mercedes-Benz Research and Development India. My work includes building REST APIs and microservices using Java and Spring Boot, with Python and Django experience from GenAI-related applications. I’ve worked with databases and cloud-based delivery, and I’m comfortable discussing API design, testing, deployment, and debugging. I’m looking for a role where I can contribute to backend systems while deepening my work across Python and Java technologies.”

Keep it honest: mention only technologies and achievements you can explain with a real example.

2. Core coding exercise: reverse words without split()

Input: `I love India`
Output 1: `India love I`
Output 2: `aidnI evol I`

Clarify the requirement: Output 1 reverses the order of words; Output 2 reverses every character in the entire sentence. These are two different operations.

Solution A — reverse the order of words, without split()

Approach: scan characters, build the current word, and store each completed word. Then read the words from the end to the beginning.

```
def reverse_word_order(s):
    words = []
    current = ""

    for ch in s:
        if ch != ' ':
            current += ch
        elif current:
            words.append(current)
            current = ""

    if current:
        words.append(current)

    result = []
    for i in range(len(words) - 1, -1, -1):
        result.append(words[i])

    return " ".join(result)

print(reverse_word_order("I love India"))
# India love I
```

Complexity: O(n) time for scanning and output; O(n) extra space for stored words. In Python, repeated string concatenation can be costly for very long strings; for an interview-sized solution this is readable, but a list-of-characters approach can avoid that concern.

Solution B — reverse every character, without slicing

```
def reverse_characters(s):
    result = ""
    for i in range(len(s) - 1, -1, -1):
        result += s[i]
    return result

print(reverse_characters("I love India"))
# aidnI evol I
```

Complexity: the loop visits n characters. Repeated immutable-string concatenation may make practical runtime worse than O(n) in some implementations; a list plus `''.join(...)` gives a robust linear-time implementation.

More efficient character reversal

```
def reverse_characters(s):
    chars = []
    for i in range(len(s) - 1, -1, -1):
        chars.append(s[i])
    return ''.join(chars)
```

Edge cases the interviewer may test

- `''` → empty string
- `'I'` → `'I'`
- `' I love India '` → decide whether to trim and collapse spaces; ask first.
- Tabs/newlines: should only literal spaces separate words, or all whitespace?

• Punctuation: `'Hello, world!'` should generally preserve punctuation with its word.

Practice variations

• Reverse each word but keep word order: `'I love India'` → `'I evol aidnI'`.

• Reverse word order in place conceptually, but preserve single spaces.

• Count words without `split()`.

• Find the first non-repeating character.

• Check whether two strings are anagrams.

3. Python coding drills with answers

Drill 1 — character frequency

Prompt: return the frequency of each character in a string.

```
def char_frequency(s):
    freq = {}
    for ch in s:
        freq[ch] = freq.get(ch, 0) + 1
    return freq
```

Pattern: hash map / dictionary. Time O(n), space O(k), where k is the number of distinct characters.

Drill 2 — palindrome check

```
def is_palindrome(s):
    left, right = 0, len(s) - 1
    while left < right:
        if s[left] != s[right]:
            return False
        left += 1
        right -= 1
    return True
```

If asked to ignore spaces, punctuation, or case, clarify the rule and normalise accordingly. Time O(n), extra space O(1) for the simple exact-character version.

Drill 3 — second largest distinct number

```
def second_largest(nums):
    largest = second = None
    for x in nums:
        if largest is None or x > largest:
            if x != largest:
                second = largest
            largest = x
        elif x != largest and (second is None or x > second):
            second = x
    return second
```

Discuss what to return if there are fewer than two distinct values. Time O(n), extra space O(1).

Drill 4 — remove duplicates while preserving order

```
def unique_in_order(items):
    seen = set()
    result = []
    for item in items:
        if item not in seen:
            seen.add(item)
            result.append(item)
    return result
```

Average time O(n), extra space O(n). This assumes items are hashable.

Drill 5 — two sum

```
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        need = target - x
        if need in seen:
            return [seen[need], i]
        seen[x] = i
    return []
```

Hash map reduces the typical brute-force $O(n^2)$ approach to O(n) time and O(n) space.

Drill 6 — SQL is not the only place to group data

Be prepared to explain how a Python dictionary groups records by a key, how a set supports membership checks, and when a list is preferable because order and duplicates matter.

4. Python, Flask, Django and FastAPI

Q1. What is the difference between Flask, Django, and FastAPI?

Flask: lightweight and flexible; you choose extensions and structure. **Django:** batteries-included framework with ORM, admin, authentication, and a conventional project structure. **FastAPI:** API-focused framework with type-hint-based validation, OpenAPI docs, and async support. Choose based on product needs, existing architecture, team familiarity, and performance requirements—not hype.

Q2. What happens when a request reaches a Django REST API?

The web server forwards the request to Django; URL routing selects a view; middleware runs around request/response processing; the view or DRF APIView/viewset performs authentication, permissions, validation, business logic, and database work; a response is returned through middleware.

Q3. What is a serializer in Django REST Framework?

It converts complex objects such as model instances into primitive data for rendering, and validates incoming primitive data before creating or updating objects. Validation should be explicit and errors should be returned clearly.

Q4. What is middleware?

A component that processes requests and/or responses across many endpoints. Common uses include authentication-related processing, request IDs, logging, CORS, and security headers. Keep it cross-cutting; business-specific logic usually belongs elsewhere.

Q5. What is the difference between sync and async endpoints?

A synchronous handler blocks its execution path while waiting. An async handler can yield control during compatible I/O waits, allowing the event loop to serve other work. Async is useful for high-concurrency I/O workloads; it does not automatically make CPU-heavy work faster.

Q6. How should an API validate input?

Validate required fields, types, ranges, formats, and business rules at the boundary. Reject invalid input with a consistent 4xx response, avoid leaking internal details, and enforce constraints again where data integrity requires it.

Q7. How do you handle errors in a REST API?

Return a stable error shape with an appropriate HTTP status, a safe message, and optionally a trace/request ID. Log the detailed exception internally. Do not expose stack traces, secrets, or raw database errors to clients.

Q8. How do you secure an API?

Use HTTPS, authentication, authorization per resource/action, input validation, rate limiting where appropriate, secure secret storage, least-privilege access, safe error responses, and audit/logging that avoids sensitive data.

REST quick reference

Method	Typical intent	Idempotent?
GET	Read a resource	Yes
POST	Create or trigger an operation	Usually no
PUT	Replace a resource	Yes
PATCH	Partially update a resource	Not guaranteed
DELETE	Delete a resource	Yes in intended effect

Common status codes: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthenticated, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable Content (framework conventions vary), 429 Too Many Requests, 500 Internal Server Error, 503 Service Unavailable.

5. Microservices and distributed systems

Q1. What is a microservice?

A small, independently deployable service organised around a business capability, with a clear boundary and ownership of its logic and data. Splitting a monolith into many services without a reason adds network, deployment, and operational complexity.

Q2. Monolith vs microservices?

A monolith can be simpler to develop, test, and deploy initially. Microservices can enable independent deployment and scaling, but require service discovery or routing, observability, resilience, distributed-data decisions, and operational maturity.

Q3. How do services communicate?

Synchronous calls such as HTTP/REST or gRPC suit request-response interactions. Asynchronous messaging such as Kafka or a queue suits decoupling, buffering, and event-driven workflows. The choice depends on latency, consistency, coupling, and delivery requirements.

Q4. What are timeout, retry, and circuit breaker?

A timeout bounds how long a caller waits. A retry repeats a transiently failed operation, ideally with bounded attempts and exponential backoff plus jitter. A circuit breaker temporarily stops calls to a failing dependency. Retries must be safe for the operation and should not amplify an outage.

Q5. How do you prevent duplicate processing?

Use idempotency keys or unique business identifiers, database uniqueness constraints, and deduplication where appropriate. For event consumers, assume messages can be delivered more than once unless the system guarantees otherwise.

Q6. What is eventual consistency?

Replicas or services may temporarily disagree after a change, but converge if updates stop and processing succeeds. It is common in distributed systems and asynchronous workflows. Explain what stale reads mean for the business.

Q7. How do you debug a slow or failing service?

Check recent deployments and error rates; correlate logs with request/trace IDs; inspect latency by dependency; review metrics for CPU, memory, database pools, queues, and timeouts; reproduce safely; then validate the fix with tests and monitoring.

Q8. What should you know about API pagination?

Offset pagination is easy and supports page numbers, but can become expensive and unstable as data changes. Cursor/keyset pagination uses an ordered key and is often more efficient for large or changing datasets. Always define deterministic ordering and sensible page-size limits.

How to explain your own backend work

Use this structure for each project: **Problem → Architecture → Your contribution → Trade-off → Result → What you would improve.**

• For Java/Spring Boot: explain a real REST endpoint, its service logic, persistence, validation, exception mapping, and tests.

• For Python/Django/GenAI work: explain the API boundary, request validation, orchestration, data flow, and how failures are handled.

• For cloud delivery: explain the CI/CD stages, configuration/secrets, deployment approach, health checks, and rollback.

• For every metric on your resume, know the baseline, measurement method, and your specific contribution.

6. Java and Spring Boot revision

Q1. What is dependency injection?

A class receives its dependencies from the framework rather than constructing them directly. It reduces coupling and improves testability. Constructor injection is generally preferred because dependencies are explicit and can be final.

Q2. Explain Controller, Service, Repository.

Controller handles HTTP concerns and maps requests/responses. Service contains business rules and coordinates work. Repository abstracts data access. Keep responsibilities separated so code is testable and changes remain local.

Q3. What is Spring Boot?

It builds on Spring with auto-configuration, starter dependencies, and embedded-server support to reduce setup. It still relies on Spring concepts such as the application context and dependency injection.

Q4. What is JPA vs Hibernate?

JPA is a persistence specification/API; Hibernate is a popular implementation. Spring Data JPA adds repository abstractions that reduce routine data-access code.

Q5. What does @Transactional do?

It defines a transaction boundary through Spring's transaction infrastructure. Know the configured propagation/isolation behavior and common proxy caveats. By default, rollback rules differ for unchecked versus checked exceptions unless configured.

Q6. ArrayList vs LinkedList?

ArrayList provides fast indexed access and usually good cache locality; inserting/removing in the middle can require shifting elements. LinkedList has costly indexed access and can insert/remove efficiently only when the relevant node/iterator position is already available. ArrayList is the usual default.

Q7. HashMap vs ConcurrentHashMap?

HashMap is not designed for concurrent mutation without external coordination. ConcurrentHashMap supports concurrent access and updates with thread-safe operations; compound workflows may still need atomic methods such as computeIfAbsent.

Q8. How do you test a Spring Boot API?

Unit-test business logic with JUnit and mocks where appropriate; use Spring test support for application integration; use MockMvc or an HTTP client for endpoint behavior; test validation, error paths, persistence, and authorization—not only happy paths.

Spring Boot request flow

```
Client
  -> HTTP request
  -> Filter / security chain
  -> DispatcherServlet
  -> Controller
  -> Service (business rules / transaction)
  -> Repository
  -> Database
  <- response / exception handler
```

Python vs Java talking points

• Python is dynamically typed and often concise; Java is statically typed and provides compile-time type checking.

• Both can build REST APIs and microservices; choose based on ecosystem, team expertise, runtime needs, and existing services.

• In Python, understand mutable vs immutable types, exceptions, iterables, context managers, and virtual environments.

• In Java, understand interfaces, collections, equals/hashCode, exceptions, streams, and concurrency basics.

7. Database and SQL questions

Example schema

```
employees(id, name, department_id, salary)
departments(id, department_name)
orders(id, customer_id, amount, created_at)
```

Q1. INNER JOIN vs LEFT JOIN

INNER JOIN returns matching rows from both sides. LEFT JOIN returns every left-side row and matching right-side rows, using NULL where there is no match.

```
SELECT e.name, d.department_name
FROM employees e
LEFT JOIN departments d ON d.id = e.department_id;
```

Q2. Find the second-highest distinct salary

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

This returns NULL if there is no second distinct salary. Be ready to discuss duplicates and NULL values.

Q3. WHERE vs HAVING

WHERE filters rows before grouping; HAVING filters groups after aggregation.

```
SELECT department_id, AVG(salary) AS avg_salary
FROM employees
WHERE salary IS NOT NULL
GROUP BY department_id
HAVING AVG(salary) > 80000;
```

Q4. What is an index?

An index speeds up suitable reads and lookups by maintaining an additional data structure, but costs storage and adds work to inserts, updates, and deletes. Index columns used selectively in filters, joins, and ordering; verify with query plans rather than indexing everything.

Q5. ACID

• **Atomicity:** all operations in the transaction happen or none do.

• **Consistency:** constraints and invariants remain valid.

• **Isolation:** concurrent transactions are controlled according to the isolation level.

• **Durability:** committed changes survive failures according to the database's guarantees.

Q6. Normalization and denormalization

Normalization reduces redundant data and update anomalies by structuring relations and dependencies. Denormalization deliberately duplicates or precomputes data to improve reads, with extra complexity to keep it consistent.

Q7. SQL vs NoSQL

Relational databases provide tables, SQL, relationships, constraints, and transactions. NoSQL is a broad set of models such as document, key-value, and wide-column stores. Pick based on access patterns, data relationships, consistency needs, scale, and operational expertise—not because one is universally faster.

Transactions: questions to expect

• What happens if one step of a multi-step update fails?

• What is a deadlock and how do you reduce its likelihood?

• Why can a missing index slow down an API?

• How would you investigate a slow query?

- Why should a database transaction not remain open during a slow external HTTP call?

8. Mock first-round interview

Answer each aloud in 60–90 seconds. For coding questions, write code and test it rather than only describing the approach.

1. **Introduce yourself and summarise your backend experience.**

   Connect your Java/Spring Boot API work with your Python/Django experience. Mention a concrete contribution and be ready to drill into it.

2. **Reverse words without split().**

   Clarify whether word order or characters should reverse. Explain the scan, storage, edge cases, and complexity.

3. **Flask vs Django vs FastAPI?**

   Compare flexibility, batteries-included features, API validation/docs, async support, and project fit.

4. **Design a REST endpoint to create a user.**

   Discuss method/path, request schema, validation, authentication, status codes, duplicate handling, persistence, and error shape.

5. **A downstream service is timing out. What do you do?**

   Use bounded timeouts, carefully scoped retries/backoff, circuit breaker where useful, observability, and safe fallback or error handling.

6. **Write SQL to find the second-highest salary.**

   Use MAX below MAX or a ranking/window-function approach; discuss ties and missing values.

7. **Explain one Spring Boot endpoint you built.**

   Trace request → controller → service → repository → DB → response, including tests and errors.

8. **How do you secure an API?**

   HTTPS, authentication, authorization, validation, rate limits, safe secrets, least privilege, and careful logging.

9. **How do you test a Python API?**

   Unit tests for business logic, endpoint/integration tests, validation and failure tests, dependency isolation, and test data cleanup.

10. **Describe a production issue you investigated.**

    Situation, symptoms, evidence, root cause, fix, verification, and prevention. Do not claim work you did not do.

11. **How would you improve a slow SQL-backed endpoint?**

    Measure first, inspect query plan, identify N+1 queries, add/select indexes, reduce payloads, paginate, and cache only where safe.

12. **Why are you interested in this role?**

    Connect the role's Python/backend stack to your current strengths and the specific skills you want to deepen.

Questions you can ask the interviewer

- What would the first few months focus on: API development, service integrations, or maintenance?
- How is the backend split between Python and Java/Spring Boot?
- What does the next interview round focus on, and should I expect another hands-on coding exercise?

9. Final revision checklist

- ■ I can reverse word order without split() and explain both requested outputs.
- ■ I tested empty input, one word, repeated spaces, and punctuation assumptions.
- ■ I can explain list vs tuple vs set vs dictionary in Python.
- ■ I can explain Flask vs Django vs FastAPI without saying one is always best.
- ■ I know GET/POST/PUT/PATCH/DELETE and common HTTP status codes.
- ■ I can explain authentication vs authorization.
- ■ I can describe timeout, retry, circuit breaker, idempotency, and pagination.
- ■ I can explain Controller/Service/Repository and dependency injection.
- ■ I can explain JPA vs Hibernate and @Transactional.
- ■ I can write a JOIN, GROUP BY/HAVING, and second-highest salary query.
- ■ I have a clear project story and can defend every resume metric.
- ■ I will think aloud, clarify assumptions, test examples, and state complexity.

A compact answer structure

Definition → **Why it matters** → **How you used it** → **Trade-off or limitation.** Example: “Pagination limits response size and database work. I’d choose cursor pagination for large, frequently changing datasets when stable ordering is available; offset pagination is simpler when page-number navigation is required.”

Tomorrow's interview behaviour

• Do not rush into code. Ask one or two clarifying questions.

• Narrate the approach, then code the simplest correct solution.

• Test a normal case and at least one edge case.

• If stuck, explain the brute-force approach first and improve it.

• When discussing a project, be precise about what you personally implemented.

• If you do not know an answer, state what you know and how you would investigate.

You do not need to know everything. You need to demonstrate sound fundamentals, readable code, honest project knowledge, and structured problem-solving.